

(Q1) QA Data Creation

Our dataset contains 138 total question-answer pairs, manually created by sampling and searching through random EECS pages. The dataset was designed to challenge the RAG system with diverse question complexity, including 90 factoid questions (65.22%), 36 multi-hop questions (26.09%), 9 yes/no questions (6.52%), and 3 counting questions (2.17%). In terms of temporal sensitivity, 36 questions (26.09%) are temporal, and 102 (73.91%) are non-temporal. The vast majority of questions, 129 (93.48%), were augmentation only. Answers are notably concise, with a mean length of 3.00 words and 97.10% of answers under 10 words. To ensure quality, we randomly sampled 40 questions for Inter-Annotator Agreement (IAA) among two team members. Agreement was 69% under a strict exact match metric, and 91% under a string-matching metric which accommodated small wording differences. All disagreements were resolved by the raters.

Sample Q&A Pairs:

1. “What is the email address of the professor who teaches the AI course in Lewis 100 and earned a BA Hons in Physics from Oxford University?” → russell@cs.berkeley.edu.
2. “How many EECS student awards include the word ‘Memorial’ in their names?” → 5.
3. “What award did Nelson Morgan receive in 2022?” → IEEE James L. Flanagan Speech and Audio Processing Award.

(Q2) Retrieval Corpus

We constructed a retrieval corpus from approximately 15,000 non-PDF, non-image URLs from the eecs.berkeley.edu and www2.eecs.berkeley.edu domains, collected over 72 hours using sitemap extraction and breadth-first crawling with robots.txt delays respected. To control corpus size, technical publications (techrpts) were limited to the 1,500 most recent from 2020–2026, and noisy listing and search-result pages were excluded. We scraped text with ResiliParse and found that lightly preserving structural HTML tags improved retrieval by retaining document-structure cues. Documents were recursively chunked with size 500 and overlap 50, embedded with BAAI/bge-base-en-v1.5, and indexed in a FAISS flat inner-product index. We evaluated corpus quality through scraping-parameter tuning, manual inspection, and downstream RAG ablations. At query time, the system retrieves 50 candidate chunks, reranks them with BAAI/bge-reranker-base, and passes the top 5 full documents to the LLM.

(Q3) RAG System

Our RAG system uses a custom three-stage pipeline: dense retrieval, cross-encoder reranking, and answer generation. For dense retrieval, we use the sentence-transformer all-MiniLM-L6-v2 with FAISS over a pre-chunked EECS corpus to retrieve an initial pool of 80 candidate chunks (first_k = 80) by dense semantic similarity. The base chunking setup uses recursive splitting with chunk size 500 and overlap 50, giving fine-grained passages while preserving local context. These 80 candidates are reranked with the BAAI/bge-reranker-base cross-encoder, and the top 5 results (k = 5) are expanded to full source-page documents before generation. This lets us rank at the chunk level while giving the generator broader, more reliable context. For answer generation, we use meta-llama/llama-3.1-8b-instruct, prompted to output exactly one short factual answer with no explanation or extra text, with a small answer budget (max_tokens = 15).

(Q4) Ablations

We conducted multiple ablations to determine the most effective RAG configuration. We selected the two ablations with the largest observed impact on performance.

Ablation	Setting Change	Results
Dense pipeline strength	Dense (no rerank, k=10) → rewritten corpus, full document, rerank (first_k=30, k=3)	F1: 0.494 → 0.534; EM: 0.400 → 0.460; Recall: 0.527 → 0.567
Chunk size / overlap	Dense (chunk_size=2000, overlap=200) → Dense (chunk_size=1000, overlap=100, with URL in header)	F1: 0.425 → 0.474; EM: 0.350 → 0.390; Recall: 0.447 → 0.508

The largest gain came from strengthening the dense retrieval pipeline, which improved F1 from 0.494 to 0.534. Using a rewritten corpus, full-document retrieval, and reranking substantially improved answer quality over a simpler dense setup without reranking. We also found that reducing chunk size from 2000 to 1000 improved performance, raising F1 from 0.425 to 0.474, which suggests that smaller chunks gave more precise retrieval context.

(Q5) Error analysis

We examined a random subset of incorrect predictions from our best-performing model, focusing on cases with F1 = 0.0, and found five main error types. Retrieval failures occurred when the model used the wrong page or section, such as predicting “Ion Stoica” instead of “Matthew Santillan” for “Who is the media contact for the EECS department?” Reasoning or aggregation errors occurred when the right page was retrieved but the wrong fact was chosen, such as “Joseph Thomas Gier” instead of “Lester Edwin Reukema” for “Who was the earliest-born professor at UC Berkeley?” Temporal errors appeared on year-specific questions, for example answering “January 2017” instead of “2016” for “At which year was RISELab established?” Answer-formatting and span-selection errors involved incomplete outputs, such as “510” instead of “1(510) 642-3214” for “What is the phone number of the EE division?” Finally, some were false negatives due to metric limitations: “Two.” versus “2” for “How many courses is Venkat Anantharam slated to teach in the academic year 2026–2027?” and “Two semesters.” versus “1 year” for “How long is the 5th years Masters programs in EECS?” To improve the metric, we would keep exact match as the main signal but add stronger normalization for number words and digits, percentages, phone numbers, dates, and simple unit equivalences such as “1 year” and “2 semesters.”

(Q6) Takeaways & Future Ideas

Our main takeaway is that strong performance in a niche-domain RAG system depends more on corpus quality and retrieval design than on the language model alone. In our experiments, many failures came from retrieving the wrong page or an unhelpful chunk, while short-answer QA also proved highly sensitive to output formatting and evaluation details. With more time, we would explore hybrid sparse+dense retrieval to better balance keyword and semantic matching, run additional ablations on chunking strategies, reranking models, and the number of retrieved candidates, and add a stronger output-normalization layer to reduce false negatives by handling cases like number words versus digits, dates, phone numbers, and simple unit equivalences.

Contribution Statement

Anthony Kuang contributed to corpus scraping, crawling, improving retrieval, submission logic, and ablations. Micah Mok contributed to corpus processing, chunking, retrieval pipeline setup (sparse, dense, hybrid), ablations, cross-encoder/re-ranking, and final model submission support. Meenakshi Mittal focused on dataset creation, retrieval ablations, error analysis, and report preparation. All team members contributed to model design discussion, analysis of results, and final writeup decisions.

GenAI statement

We used GenAI primarily for debugging, tab autocomplete (with ability to replicate), and helping with API usage, in accordance with course policy.